Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	
Student Name	

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):            # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):         # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in
km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and

repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Code of Conduct

I **M Sivam Kumar (192521017)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *sivam* Date: 24/10/26

Corrected Python Program

```
import math

def sort_alerts(alerts):

"""Stable insertion sort: highest severity first."""

for i in range(1, len(alerts)):

key = alerts[i]

j = i - 1

while j >= 0 and alerts[j][1] < key[1]:

alerts[j + 1] = alerts[j]

j -= 1

alerts[j + 1] = key

return alerts

def is_blocked(blocklist, ip):

"""Return True when the IP appears in the blocklist."""

for i in range(len(blocklist)):

if blocklist[i] == ip:

return True

return False

def find_signature(log, pattern):

"""Return every zero-based start index of a nonempty pattern."""

n, m = len(log), len(pattern)

hits = []

if m == 0:

return hits

for i in range(n - m + 1):

match = True

for j in range(m):

if log[i + j] != pattern[j]:

match = False
```

```
break
if match:
hits.append(i)
return hits
def closest_logins(events):
"""Return indices and Euclidean distance of closest distinct events."""
if len(events) < 2:
return None, math.inf
min_dist = math.inf
pair = None
for i in range(len(events)):
for j in range(i + 1, len(events)):
d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
(events[i][1] - events[j][1]) ** 2)
if d < min_dist:
min_dist = d
pair = (i, j)
return pair, min_dist
alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
print("Sorted Alerts:", sort_alerts(alerts))
for ip in queries:
print(ip, ":", is_blocked(blocklist, ip))
print("Signature Positions:", find_signature(log, pattern))
print("Closest Login Pair:", closest_logins(events))
# Boundary tests
assert is_blocked([], "10.0.0.5") is False
assert find_signature("ABCFAIL", "FAIL") == [3]
assert find_signature("ABC", "ABCDE") == []
```

```
assert find_signature("ABC", "") == []

assert sort_alerts([("A1", 5), ("A2", 5)]) == [("A1", 5), ("A2", 5)]

assert closest_logins([(2, 3)]) == (None, math.inf)

assert closest_logins([]) == (None, math.inf)

print("All boundary tests passed.")
```

Expected Output

```
Sorted Alerts: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

10.0.0.9 : True

10.0.0.5 : True

8.8.8.8 : False

Signature Positions: [0, 8]

Closest Login Pair: ((0, 5), 1.4142135623730951)

All boundary tests passed.
```

Corrections Made

Sorting: fixed loop boundary and insertion index, preserving equal-severity arrival order.

Blocklist: return True for a match; return False only after searching the whole list.

Signature matching: include the last alignment and require a complete match.

Closest pair: initialise minimum to infinity, compare distinct pairs once, and choose the smallest distance.

Note: This is the debugged brute-force program. The separate production optimisation and benchmarking requirements are not included here.